Project Interim Report

On

File Compression Simulator
(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming With Java

SESSION: 2025-26

Submitted By:

Rishu Raj Kr.

RU-25-11178

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with Google

Under the Guidance of

Mr. Ashish Shivare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

Abstract

The Grocery Billing Receipt Generator is a Java-based application developed using Object-Oriented Programming (OOP) principles. The project simulates a real-world retail billing system where customers purchase multiple grocery items and receive a formatted receipt. It is needed because manual billing is error-prone and time-consuming; an automated system ensures accuracy and efficiency. The system stores product details such as name, price, and quantity using an **Item** class, and aggregates multiple items using a **Bill** class. It automatically calculates the subtotal, applies a 5% GST tax, and generates a clean, well-structured receipt using **StringBuilder**. This project demonstrates key OOP concepts including object composition, encapsulation, and internal calculation logic, making it a practical application of Java programming in the retail domain.

Introduction

Retail billing systems are fundamental tools in modern commerce. This project focuses on building a **Grocery Billing Receipt Generator** that models the operations of a simple billing counter using Java and OOP. The system manages product entries, computes totals with tax, and presents results in a readable receipt format.

Features of the project include:

- Storing product details (name, price, quantity) via an **Item** class
- Aggregating multiple items using a **Bill** class
- Automatic subtotal and tax (5% GST) calculation
- Formatted receipt generation using **StringBuilder**
- Clean and structured console output

Objectives

- Store and manage grocery item records (name, price, quantity)
- Automatically calculate total cost and applicable tax
- Generate a well-formatted billing receipt
- Apply OOP concepts such as encapsulation and aggregation
- Demonstrate efficient string handling using StringBuilder

Scope

This project is suitable as a learning exercise for understanding Java OOP concepts in a real-world retail billing scenario. It can be extended in the future with database integration, GUI interfaces, discount management, and barcode scanning functionality.

System Requirements

Hardware: Computer with 4GB RAM

Software: Java JDK, IDE (VS Code / Eclipse / Edit Plus)

Methodology

The system works as follows: First, multiple **Item** objects are created with their name, price, and quantity. These items are then passed as an array to the **Bill** class. The Bill class loops through all items to compute the subtotal for each (price × quantity) and sums them to find the total. A 5% tax is then applied to the total to compute the grand total. Finally, the **generateReceipt()** method uses a **StringBuilder** to construct and display a neatly formatted receipt listing each item's details, total, tax, and grand total.

Flowchart & Class Diagram

The flowchart begins at Start, proceeds to item entry, then loops through all items calculating subtotals, sums to a total, applies tax, formats the receipt using StringBuilder, prints the receipt, and ends.

Sample Class Diagram:

Item

o String name

o double price

o int quantity

o Item(String name, double price, int quantity)

o getName(), getPrice(), getQuantity()

Bill

o Item[] items <-- array of Item objects (aggregation)

o double total, tax, grandTotal

o Bill(Item[] items)

o calculateTotal()

o generateReceipt()

Implementation

The project consists of two primary classes:

Item Class: This class encapsulates all attributes of a single grocery product. It stores the item's name, price per unit, and quantity purchased. The constructor initialises all three fields, and getter methods provide read-only access to them from outside the class.

Bill Class: This class accepts an array of Item objects via its constructor, demonstrating object composition (aggregation). The *calculateTotal()* method iterates through all items, multiplying each item's price by its quantity to get the subtotal, and accumulates these into a grand total. Tax is computed as 5% of the total. The *generateReceipt()* method builds a formatted string using StringBuilder, appending each item's row and the summary lines (Total, Tax, Grand Total), and finally prints the receipt.

Sample Input / Output

Input (main method):

```
Item[] items = {
new Item("Rice", 50, 2),
new Item("Milk", 30, 3)
};
Bill bill = new Bill(items);
bill.generateReceipt();
```

Output (Terminal):

```
------ Grocery Bill ------
Item Price Qty Total
Rice 50 2 100
Milk 30 3 90
Total: 190.0
Tax (5%): 9.5
Grand Total: 199.5
```

Future Enhancements

- Add a GUI (Swing/JavaFX) for a graphical billing interface
- Integrate with a database (MySQL/SQLite) to store transaction history
- Support multiple tax rates (GST slabs: 5%, 12%, 18%)
- Add discount and coupon code functionality
- Generate PDF receipts for customers
- Barcode/QR code scanning for item entry

Gantt Chart

1. Planning & Structure

A Gantt chart helps break the project into tasks (design, coding, testing, etc.) and set timelines. Even if already built once, rebuilding or modifying still needs planning.

2. Time Management

It shows:

- When each task starts
- When it should finish
- This helps avoid delays and last-minute panic.

Task	Week 1	Week 2	Week 3	Week 4
Requirement Analysis	✓			
Class Design (Item, Bill)	✓	✓		
Implementation / Coding		✓	✓	
Testing & Debugging			✓	✓

Report Writing			✓	✓
Final Submission				✓

Conclusion

The Grocery Billing Receipt Generator project successfully demonstrates key Java OOP concepts including encapsulation, object composition (aggregation), and internal calculation logic. By implementing the Item and Bill classes along with StringBuilder-based receipt formatting, the project provides a functional and structured billing system. It reflects a real-world application of OOP principles and serves as a strong foundation for more advanced retail system development.

References

Book

1 H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

Website

2 Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com

Journal Article

3 A. Sharma and R. Gupta, "Object-Oriented Design Patterns in Java," International Journal of Computer Science, vol. 8, no. 1, pp. 12-18, 2021.

Conference Paper

4 S. Kumar, "Teaching OOP Through Real-World Applications," in Proc. IEEE Conf. on Computer Science Education, 2022, pp. 55-60.